



ORIGIN PROTOCOL

Vanilla Compounding Staking Strategy

Security Assessment Report

Version: 2.0

June, 2026

Contents

Introduction	2
Disclaimer	2
Document Structure	2
Overview	2
Security Assessment Summary	3
Scope	3
Approach	3
Coverage Limitations	3
Findings Summary	3
Detailed Findings	4
Summary of Findings	5
Missing NON_REGISTERED State Check In registerSsvValidator()	6
Miscellaneous General Comments	8
A Vulnerability Severity Classification	10

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Origin Protocol components in scope. The review focused solely on the security aspects of the Solidity implementation of the contracts, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Origin Protocol components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Origin Protocol components in scope.

Overview

Origin Protocol is a suite of DeFi products designed to increase economic opportunity through innovative blockchain technologies, including liquid staking tokens and yield-generating stablecoins.

This review covers the refactoring of Origin's compounding staking infrastructure to remove its dependency on SSV Network. The shared compounding validator logic is extracted into a new standalone `CompoundingStakingStrategy` that supports vanilla (non-SSV) validators, while SSV-specific behaviour is retained in a slimmed-down `CompoundingStakingSSVStrategy`.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [OriginProtocol/origin-dollar](#) repository.

The scope of this time-boxed review was strictly limited to files changed in PRs [#2905](#) and [#2885](#), with particular focus on `CompoundingStakingStrategy.sol`. Changes made to `ConsolidationController.sol`, `NativeStakingSSVStrategy.sol`, and `CompoundingStakingSSVStrategy.sol` were not in scope.

The fixes of the identified issues were assessed at commit [49609a2](#).

Note: third party libraries and dependencies were also excluded from the scope of this assessment.

Approach

The security assessment covered components written in Solidity.

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: <https://github.com/Cyfrin/aderyn>
- Slither: <https://github.com/trailofbits/slither>
- Mythril: <https://github.com/ConsenSys/mythril>

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 2 issues during this assessment. Categorised by their severity:

- Informational: 2 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Origin Protocol components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each finding by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open**: the finding has not been addressed by the project team.
- **Resolved**: the finding was acknowledged by the project team and updates to the affected components have been made to mitigate the related risk.
- **Closed**: the project team has reviewed and acknowledged the finding, but the recommended remediation has not been implemented. The project team has typically provided a documented rationale supporting this decision, including the approach taken and any associated risk considerations.

Summary of Findings

ID	Description	Severity	Status
ORGN4-01	Missing NON_REGISTERED State Check In registerSsvValidator()	Informational	Resolved
ORGN4-02	Miscellaneous General Comments	Informational	Closed

ORGN4-01	Missing NON_REGISTERED State Check In registerSsvValidator()
Assets	contracts/contracts/strategies/NativeStaking/CompoundingStakingSSVStrategy.sol
Status	Resolved: See Resolution
Rating	Informational

Description

The `registerSsvValidator()` function dropped the explicit `NON_REGISTERED` state precondition present in the original `CompoundingValidatorManager`, shifting duplicate-registration prevention entirely to the external SSV Network contract.

The pre-refactor implementation guarded registration with a local state check:

```
contracts/contracts/strategies/NativeStaking/CompoundingValidatorManager.sol::CompoundingValidatorManager
::registerSsvValidator()

require(
    validator[pubKeyHash].state == ValidatorState.NON_REGISTERED,
    "Validator already registered"
);
```

The new `CompoundingStakingSSVStrategy.registerSsvValidator()` unconditionally writes `validator[pubKeyHash].state = ValidatorState.REGISTERED` before calling the SSV Network:

```
contracts/contracts/strategies/NativeStaking/CompoundingStakingSSVStrategy.sol
::CompoundingStakingSSVStrategy::registerSsvValidator()

function registerSsvValidator(
    bytes calldata publicKey,
    uint64[] calldata operatorIds,
    bytes calldata sharesData,
    Cluster calldata cluster
) external payable onlyRegistrator whenNotPaused {
    bytes32 pubKeyHash = _hashPubKey(publicKey);

    // @audit no check that validator state is NON_REGISTERED

    validator[pubKeyHash].state = ValidatorState.REGISTERED;

    ISSVNetwork(SSV_NETWORK).registerValidator{ value: msg.value }(
        publicKey,
        operatorIds,
        sharesData,
        cluster
    );
}
```

The SSV Network's `ValidatorLib.registerPublicKey()` reverts with `ValidatorAlreadyRegistered` for any public key with an existing live registration, which prevents the state overwrite from persisting for validators in states `REGISTERED` through `EXITING`. However, the SSV Network's `_bulkRemoveValidator()` deletes the validator's storage slot (`delete s.validatorPKs[hashedValidator]`), which means a pubkey can be re-registered after removal. Since the Origin strategy's `removeSsvValidator()` calls `ISSVNetwork.removeValidator()`, any validator that reaches the `REMOVED` state can subsequently be re-registered on SSV without reverting, allowing the local state to be overwritten from `REMOVED` to `REGISTERED`.

This issue is rated as informational as the function is registrator-gated and the `REMOVED` state is a terminal state in the lifecycle. Nonetheless, the local state machine's correctness for non-terminal states depends on an external contract's behaviour rather than being self-enforcing.

Recommendations

Restore the local `NON_REGISTERED` precondition so the strategy's state machine does not depend on an external contract for correctness. Use a custom error to remain within EIP-170 size constraints.

Resolution

The development team has added a check to the `registerSsvValidator()` function to ensure that the validator state is `NON_REGISTERED`.

This issue has been resolved in PR [#2915](#).

ORGN4-02	Miscellaneous General Comments	
Assets	/*	
Status	Closed: See Resolution	
Rating	Informational	

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. Inaccurate Comment On NON_REGISTERED Validator State

Related Asset(s): `contracts/contracts/strategies/NativeStaking/CompoundingStakingStrategy.sol`

The inline comment on `ValidatorState.NON_REGISTERED` conflates staking with beacon-chain deposit receipt. A validator leaves `NON_REGISTERED` when its first deposit is staked via `stakeEth()` and `_recordFirstDeposit()`, at which point state becomes `STAKED`. Receiving the deposit on the beacon chain when the pending deposit is processed is a later phase covered by `VERIFIED`.

```
contracts/contracts/strategies/NativeStaking/CompoundingStakingStrategy.sol
::CompoundingStakingStrategy

enum ValidatorState {
    NON_REGISTERED, // validator has not received its first deposit // @audit inaccurate: should refer to staking, not
                    ↪ beacon-chain receipt
    // ... snip ...
    STAKED, // validator has funds staked
    VERIFIED, // validator has been verified to exist on the beacon chain
}
```

Update the `NON_REGISTERED` comment to refer to staking rather than beacon-chain receipt:

```
NON_REGISTERED, // validator has not staked its first deposit
```

2. Inconsistent Error Handling In CompoundingStakingStrategy

Related Asset(s): `contracts/contracts/strategies/NativeStaking/CompoundingStakingStrategy.sol`

The `CompoundingStakingStrategy` contract uses a mix of custom error types and string-based `require` statements for error handling, creating an inconsistent codebase.

The contract defines six custom errors:

```
contracts/contracts/strategies/NativeStaking/CompoundingStakingStrategy.sol
::CompoundingStakingStrategy

error NotRegistratorOrGovernor(); // 0xbf454a2d
error NotRegistrator(); // 0x929dfit920
error NoFirstDeposit(); // 0x30e60c37
error InvalidFirstDepositAmount(); // 0x29829bdd
error UnsupportedAsset(); // 0x24a01144
error UnsupportedFunction(); // 0xea1c702e
```

However, the majority of error conditions use string-based `require` statements instead. For example:

```
contracts/contracts/strategies/NativeStaking/CompoundingValidatorManager.sol
::CompoundingValidatorManager::stakeEth()

require(
    depositAmountWei <= IWETH9(WETH).balanceOf(address(this)),
    "Insufficient WETH"
);
require(depositList.length < MAX_DEPOSITS, "Max deposits");
// ... snip ...
require(depositAmountWei >= 1 ether, "Deposit too small");
```

Convert all string-based `require` statements to custom error types with `if/revert` patterns. This improves gas efficiency, reduces deployed bytecode size, and provides a consistent developer experience for offchain error handling.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team's responses to the raised issues above are as follows.

1. Updated the comment on the `NON_REGISTERED` validator state to refer to staking rather than beacon-chain receipt in PR [#2916](#).
2. Acknowledged with the following comment:

"No changes here as the other vault strategies use string errors. The custom errors introduced in the staking strategy contract are isolated to staking strategy specific errors. Common string errors that are also thrown by other strategies have been left. The custom errors were required to get the contract to a deployable size."

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'